ROBOTICS SOFTWARE STATE OF PRACTICE

A STATE-OF-THE-PRACTICE STUDY OF ROBOTICS SOFTWARE FOR MOTION PLANNING AND SIMULATION

BY

ZIYANG FANG, B.Eng.

A Report

submitted to the Department You Belong To

and the School of Graduate Studies

of McMaster University

in partial fulfilment of the requirements

for the degree of

Masters of Engineering

Masters of Engineering (YYYY) McMaster University

(Department You Belong To) Hamilton, Ontario, Canada

TITLE:	A State-of-the-Practice Study of Robotics Software for Motion Planning and Simulation
AUTHOR:	Ziyang Fang B.Eng. (Software Engineering & Game Design), McMaster University, Hamilton, Canada
SUPERVISOR:	Spencer Smith
NUMBER OF PAGES:	xiv, **??**

Lay Abstract

Abstract

Your Dedication

Optional second line

Acknowledgements

Contents

List of Figures

List of Tables

Notation, Definitions, and Abbreviations

Notation

$A \leq B$	A is less than or equal to B

Definitions

Challenge	With respect to video games, a challenge is a set of goals presented to the player that they are tasks with completing; challenges can test a variety of player skills, including accuracy, logical reasoning, and creative problem solving

Abbreviations

AI	Artificial intelligence

Declaration of Academic Achievement

The student will declare his/her research contribution and, as appropriate,

Chapter 1

Introduction

Robotics software for motion planning and simulation enables researchers and engineers to model robot kinematics and dynamics, simulate sensor data, plan collision-free trajectories, and test control strategies before deploying to physical hardware. These software systems are not peripheral utilities; in many projects they determine which experiments are feasible, whether results can be reproduced, and whether an idea moves from simulation to a real robot. Their quality and maintainability shape both the pace of research and the reliability of practical applications.

The robotics software ecosystem includes a diverse range of tools: full simulation environments, physics engines, dynamics libraries, motion planning libraries, middleware frameworks, and supporting infrastructure. This diversity is valuable in practice, but it complicates comparison. A researcher or engineer selecting software for a robotics workflow may need to evaluate installation effort, documentation quality, repository activity, testing practices, and whether a tool's capabilities match the intended task—questions that belong as much to software engineering as to robotics.

The gap between software engineering best practices and research software development is well documented. Scientists and engineers developing research software often learn software engineering skills informally, from peers, through self-study, or by trial and error, rather than through formal training **?**. Hannay et al. **?** observe that many scientists show indifference to standard software engineering concepts. Prabhu et al. **?** report that more than half of the scientists they surveyed did not use a debugger. Because much of the software studied in this thesis originates in research settings, these concerns about development practices apply directly to the domain.

This thesis conducts a State-of-the-Practice study of robotics software for motion planning and simulation. It does not propose a new robotics algorithm, implement a simulator, or benchmark robot performance. Its purpose is to collect and organize evidence about existing software packages in the domain, using public artifacts such as repositories, documentation, issue trackers, publications, and project metadata.

1.1 Research Questions

The research questions follow the State-of-the-Practice methodology described by Smith et al. **?** and are adapted to the domain of robotics software for motion planning and simulation:

RQ1. What software packages are candidates for a State-of-the-Practice study of robotics software for motion planning and simulation, and how can the candidate set be constructed and justified transparently?

RQ2. What artifacts, tools, and publicly visible development practices are present in the selected software packages, and what do they reveal about the state of

practice in this domain?

RQ3. What commonalities, variabilities, and parameters of variation characterize the selected software packages when treated as a software family?

RQ4. What public evidence can be collected for each of the Domain X measurement categories: installability, correctness and verifiability, surface reliability, surface robustness, surface usability, maintainability, reusability, surface understandability, and visibility and transparency?

RQ5. If approved developer interview data becomes available, what developer pain points and quality-assurance practices are reported for this domain?

Together, these questions separate the construction of the study set from the measurement and interpretation work. RQ1 covers candidate discovery and filtering. RQ2 and RQ4 cover the public evidence collected through the measurement template. RQ3 covers the commonality analysis. RQ5 remains conditional until approved interview material is available.

1.2 Scope

The working domain for this thesis is *robotics software for motion planning and simulation*. This domain was established through discussion with the supervisor and the domain expert, Dr. Matthew Giamou. It was initially discussed in relation to inverse kinematics software, but later project communication settled on the broader domain used here. The scope includes robotics simulators and closely related software used for planning, dynamics, physics, middleware, and robotic system development.

Several important and widely used tools fall outside the scope of this study. MATLAB and Simulink are major platforms in both robotics research and industry, but they are proprietary and do not permit the kind of public repository-based measurement that this study depends on. Dr. Giamou confirmed that excluding MATLAB and Simulink does not misrepresent the domain. Unity, NVIDIA Isaac Sim, and RaiSim were excluded because their core simulation components are not open source, which prevents consistent repository mining across the study set. MRDS and USARSim were excluded because they are no longer actively maintained and do not represent current practice. GPMP2 and CHOMP, while relevant to optimization-based motion planning, are more narrowly scoped than the general-purpose tools in the candidate list and were excluded to maintain methodological consistency.

The study depends on evidence from public repositories, documentation, issue trackers, publications, and project metadata. Open-source availability and repository-based measurability are therefore essential selection criteria. A tool may be important in robotics practice and still fall outside the measured set if its core components are proprietary, if public repository evidence is unavailable, or if the project is no longer maintained enough to represent current practice.

The selected set is deliberately heterogeneous. Many entries are full robotics simulators, but ROS 2, MoveIt!, OMPL, and similar tools serve different roles. They remain in scope because motion planning and simulation workflows often depend on middleware, planning libraries, and supporting infrastructure. The comparison therefore treats the selected packages as related members of a domain, not as interchangeable products.

1.3 Overview of the Methodology

The methodology follows the State-of-the-Practice approach developed by Smith et al. **?**, which has been applied to domains such as geographic information systems, mesh generation, seismology software, statistical software for psychology, medical imaging software, and Lattice Boltzmann method software.

Candidate software packages were identified from three types of sources: academic survey papers, community-curated software lists, and LLM-generated software lists. The LLM-generated lists were used as an auxiliary discovery source, not as the sole basis for candidate selection. They were combined with the academic and community-curated sources to produce a broad initial pool.

Each package appearing in the collected sources was counted once per source to compute a mention count. The mention count is a source-appearance signal: a rough indication of how commonly each package is referenced across the collected sources. It is not a measure of software quality, adoption, research impact, or fitness for purpose. The candidates were ranked by mention count, and the list was filtered using open-source availability, repository-based measurability, and maintenance status to produce a final study set of 28 packages.

The selected packages are examined in three main ways. First, the Domain X measurement template organizes repository-based and public evidence into summary information, repository metrics, and nine quality categories drawn from Appendix B of the methodology. Second, a commonality analysis describes the shared capabilities, variabilities, and parameters of variation across the selected packages, treating them as members of a software family. Third, if approved interview data becomes available, semi-structured developer interviews can add a practitioner perspective on

development pain points and quality-assurance practices.

1.4 Contributions

This thesis is intended to make the following contributions:

1. A scoped State-of-the-Practice study for robotics software for motion planning and simulation, applying and extending a peer-reviewed methodology to a new domain.

2. A documented and transparent candidate discovery process that combines academic, community-curated, and LLM-assisted sources, with explicit discussion of the strengths and limitations of each source type.

3. An explanation of how mention count, open-source availability, and repository-based measurability are used together to construct and justify the final study set.

4. A measurement dataset covering 28 software packages across nine quality categories, collected from public repositories and project artifacts and organized according to the Domain X Appendix B template.

5. A commonality analysis that identifies shared capabilities, dimensions of variation, and parameters of variation across the selected packages, supporting comparison beyond numeric scores alone.

These contributions are methodological and descriptive. The thesis investigates existing software; it does not assume that any single package is best for all robotics tasks, nor does it aim to replace community judgement with a formulaic ranking.

1.5 Organization

The remainder of this thesis is organized as follows:

- **Chapter 2** introduces the robotics software domain, reviews the State-of-the-Practice methodology and its prior applications, defines the software quality attributes used in the study, and provides background on commonality analysis.
- **Chapter 3** describes the full methodology: domain selection, candidate discovery from multiple source types, mention-count ranking, filtering criteria, the measurement procedure using the Domain X template, commonality analysis procedure, and the planned interview component.
- **Chapter 4** presents the 28 selected software packages with summary metadata, and reports the commonality analysis: shared capabilities, variabilities, and parameters of variation.
- **Chapter 5** reports the measurement results organized by the nine Domain X quality categories, including repository metrics and overall observations.
- **Chapter 6** describes the developer interview component, including recruitment, protocol, and analysis plan, and reports findings if approved interview data is available.
- **Chapter 7** answers the research questions, interprets the measurement results, compares findings with prior State-of-the-Practice studies, and discusses threats to validity.
- **Chapter 8** presents recommendations for users and maintainers of robotics software, and for future State-of-the-Practice studies.

- **Chapter ??** summarizes the study, its key findings, limitations, and directions for future work.

Chapter 2

Background

This chapter provides the foundational concepts needed to understand the methodology, measurements, and analysis that follow. Section 2.1 introduces the domain of robotics software for motion planning and simulation. Section 2.2 defines the software categories relevant to this study. Section 2.3 reviews the State-of-the-Practice methodology and its prior applications. Section 2.4 defines the nine software quality attributes measured in this study. Section 2.5 introduces commonality analysis as a method for describing software families. Section 2.6 discusses the role and limitations of open-source repository-based measurement.

2.1 Robotics Software for Motion Planning and Simulation

Robotics software for motion planning and simulation supports the design, testing, evaluation, and deployment of robotic systems. Motion planning software generates

feasible robot motions under constraints such as geometry, kinematics, dynamics, collisions, and task requirements **?**. Simulation software provides virtual environments for representing robots, sensors, actuators, controllers, and physical interactions before, or alongside, work with physical hardware **?**.

The domain in this thesis is broader than a single class of software. It includes full simulation environments (such as Gazebo, Webots, CoppeliaSim, and CARLA), physics and dynamics engines (such as Bullet, MuJoCo, ODE, and DART), motion planning libraries (such as OMPL), manipulation frameworks (such as MoveIt!), middleware (such as ROS 2, YARP, and OpenRTM-aist), and related development tools. These systems often appear together in a robotics workflow even when they are not direct substitutes for one another. A simulator may depend on a physics engine for contact and dynamics computation. A planning stack may use a collision-checking library and communicate through middleware. Middleware may connect planning, control, perception, and simulation components into an integrated system.

This breadth creates a scope challenge for a State-of-the-Practice study. A narrow definition restricted to full simulators would make comparison simpler, but it would exclude software that practitioners routinely use in motion planning and simulation workflows. A broader definition better reflects the practical ecosystem, provided the thesis is explicit about the different roles that software packages play within the domain.

2.2 Software Categories

When assessing software packages, it is necessary to understand the licensing model under which each package is distributed. In particular, the availability of source code

determines whether repository-based measurement is possible. Following the terminology used in prior State-of-the-Practice studies **??**, we distinguish three common software categories:

Open Source Software (OSS): For OSS, the source code is openly accessible. Users have the right to study, change, and distribute it under a license granted by the copyright holder. For many OSS projects, the development process relies on collaboration among contributors worldwide. Accessible source code exposes the underlying logic of software functions, the development practices used to produce them, and the flaws and potential risks in the final product. OSS is therefore suitable for the kind of repository-based analysis performed in this study.

Freeware: Freeware is software that can be used free of charge. Unlike OSS, the authors of freeware do not permit access to or modification of the source code. To many end users, the distinction between freeware and OSS may not matter. However, for researchers seeking insight into software development processes, inaccessible source code is a barrier to measurement.

Commercial Software: Commercial software is software developed by a business as part of its business. Typically, commercial software requires payment to access all features and does not include access to the source code. Some commercial software is available free of charge, but the lack of source access prevents the kind of repository-based assessment used in this study.

This study assesses only software that fits under the Open Source Software category. This restriction is a methodological necessity, not a judgement about the quality

or importance of freeware or commercial tools. The implications of this restriction for the composition of the study set are discussed in Chapter 3.

2.3 State-of-the-Practice Studies

A State-of-the-Practice study examines existing tools, artifacts, processes, and evidence in a chosen domain. Unlike a conventional implementation thesis, the goal is not to design a new system but to learn from existing software by systematically collecting and organizing public evidence. Claims about development activity, documentation quality, issue management, installation behaviour, or maintainability must be supported by observable project artifacts. When evidence is missing or ambiguous, the limitation is recorded rather than filled with assumptions.

The methodology used in this thesis is based on "Methodology for Assessing the State of the Practice for Domain X" by Smith et al. **?**. The methodology is designed to be generic—applicable to any research software domain—and feasible for a small team working within approximately 160 person-hours per domain. It prescribes a systematic process: identify a suitable domain, engage a domain expert, construct and filter a candidate software list, perform domain analysis, gather source code and documentation, collect repository metrics, apply a measurement template covering nine quality categories, survey developers through semi-structured interviews (where ethics approval permits), use the Analytic Hierarchy Process (AHP) to support ranking, and answer a set of research questions.

This methodology builds on prior State-of-the-Practice assessments conducted across several research software domains: Geographic Information Systems **?**, Mesh Generators **?**, Seismology software **?**, and Statistical software for psychology **?**. More

recently, the methodology was refined and applied to Medical Imaging (MI) software **?** and Lattice Boltzmann Method (LBM) software **?**. These two studies serve as the primary structural and stylistic references for this thesis.

The MI and LBM studies follow a common organizational pattern: introduction and motivation, background on the domain and methodology, detailed methodology description, measurement results organized by quality category, comparison with community rankings, developer interviews and pain points, discussion of research questions, recommendations, and conclusions. This thesis adopts the same general structure while adapting the content to the domain of robotics software for motion planning and simulation.

Key features of the Domain X methodology that distinguish it from ad hoc software comparisons include: (i) the involvement of a domain expert to vet the software list, domain analysis, and ranking; (ii) a transparent candidate discovery process using multiple source types; (iii) a standardized measurement template that applies the same 108 questions to every package; (iv) the use of repository-based metrics alongside qualitative assessments; and (v) a commonality analysis that treats the selected packages as members of a software family rather than as unrelated products.

2.4 Software Quality Attributes

Quality is defined as a measure of the excellence or worth of an entity. As is common practice in software engineering, we do not treat quality as a single measure but as a collection of distinct qualities, often called "ilities" **?**. This study assesses nine quality attributes drawn from the Domain X Appendix B measurement template. Several of the qualities use the word "surface" to emphasize that the assessment is based

on shallow but systematic checks performed within a limited time budget, not on exhaustive experimental evaluation **?**.

Installability: The effort required for the installation and uninstallation of software in a specified environment. Measured through the presence and quality of installation instructions, the number of steps required, the availability of automation, the handling of dependencies, and the presence of validation procedures.

Correctness and Verifiability: A program is correct if it matches its specification, which may be stated explicitly or implicitly. The related quality of verifiability is the ease with which the software components or the integrated product can be checked to demonstrate correctness. Measured through the presence of requirements or theory documentation, techniques used to build confidence in correctness (such as automated testing, assertions, or continuous integration), and the availability and quality of tutorials with expected outputs.

Surface Reliability: The probability of failure-free operation of a computer program in a specified environment for a specified time. Surface reliability is assessed through whether the software breaks during installation or initial tutorial use, whether descriptive error messages are displayed, and whether recovery is possible.

Surface Robustness: Software possesses robustness if it behaves reasonably when it encounters circumstances not anticipated in the requirements specification, and when users violate the assumptions in its requirements specification. Surface robustness is assessed through whether the software handles unexpected or malformed input gracefully, including edge cases such as modified line endings

in text input files.

Surface Usability: The extent to which a product can be used by specified users to achieve specified goals with effectiveness, efficiency, and satisfaction in a specified context of use. Surface usability is assessed through the presence of getting-started tutorials, user manuals, documented user characteristics, and the availability and variety of user support channels.

Maintainability: The effort with which a software system or component can be modified to correct faults, improve performance or other attributes, or satisfy new requirements. Assessed through version information, contribution and review guidance, available non-code artifacts, issue tracking practices, the percentage of identified issues that are closed, the percentage of code that consists of comments, and the version control system in use.

Reusability: The extent to which a software component can be used with or without adaptation in a problem solution other than the one for which it was originally developed. Assessed through the number of code files and the presence and quality of API documentation.

Surface Understandability: The capability of the software product to enable the user to understand whether the software is suitable, and how it can be used for particular tasks and conditions of use. Surface understandability is assessed through a review of randomly selected source files, examining indentation consistency, coding standards, identifier quality, use of named constants, comment clarity, algorithm references, parameter ordering, and modularization.

Visibility and Transparency: The extent to which all steps of a software development process and the current status of that process are conveyed clearly to external observers. Assessed through whether the development process is defined and documented, whether the development environment is documented, and whether release notes are published.

These nine qualities organize public evidence into comparable categories. They are not complete evaluations of each package. Scores may vary with the use case, platform, user background, or measurement date. Later chapters therefore report both the measurements and their limitations.

2.5 Commonality Analysis

A domain analysis consists of systematically identifying and documenting the commonalities, variabilities, and parameters of variation of a software family **?**. A software family is defined as "a set of programs whose common properties are so extensive that it is advantageous to study the common properties of the programs before analyzing individual members" **?**. Weiss **?** defines commonality analysis as an approach to defining a family by identifying three kinds of information:

Commonalities: Goals, theories, models, definitions, and assumptions that are shared across family members. For example, all robotics simulators share the goal of representing robot kinematics and dynamics in a virtual environment.

Variabilities: Goals, theories, models, definitions, and assumptions that differ between family members. For example, simulators may differ in the physics engines they use, the sensor models they support, or the robot platforms they can

represent.

Parameters of Variation: The possible values for each variability, along with their binding time. Binding time records when a particular variation is fixed: at specification time (when the software is designed), at build time (when the software is compiled), at configuration time (when the software is set up for use), or at run time (when the software is executing).

Commonality analysis was added to the Domain X methodology after early applications revealed that software packages within a domain can vary considerably, even when they appear to belong to the same family **?**. Without a commonality analysis, comparisons risk treating fundamentally different types of software as if they were interchangeable. The analysis makes heterogeneity explicit and provides a framework for interpreting measurement results in the context of each package's role.

In this thesis, commonality analysis is particularly important because the selected set is deliberately heterogeneous: it includes full simulators, physics engines, planning libraries, and middleware. The commonality analysis identifies shared concerns across these categories (such as robot model representation, integration with external tools, and documentation practices) while also documenting the dimensions on which packages differ.

2.6 Open Source and Repository-Based Measurement

The measurement process in this study depends on evidence from public artifacts. Public repositories can reveal commit history, release activity, issue tracking, documentation, source code organization, testing practices, dependencies, and community interaction. Public documentation and project websites can add evidence about installation procedures, usage patterns, supported platforms, and design goals. This dependence on public evidence makes open-source availability a prerequisite for inclusion in the study set.

Repository-based measurement has inherent limitations. Repository metadata represents a snapshot in time: last commit dates, issue counts, release information, and pull request activity can change after measurement. Some projects distribute their development activity across multiple repositories or use external issue trackers, making it difficult to capture a complete picture from any single repository. The interpretation of repository metrics requires care: a high number of open issues may indicate an active user community rather than poor maintenance, and a low commit frequency may reflect project maturity rather than abandonment.

For these reasons, the measurement record must include clear documentation of measurement dates, repository choices, and interpretation rules. The dependence on public evidence also affects the composition of the study set. Proprietary tools, or tools with closed core components, may be important in robotics practice but cannot be measured with the same procedure as open-source tools. Chapter 3 discusses the specific inclusion and exclusion decisions that result from this constraint.

Chapter 3

Methodology

This chapter describes the methodology used to conduct the State-of-the-Practice study of robotics software for motion planning and simulation. The methodology follows Smith et al. **?** and adapts it to the current domain. Section 3.1 describes the domain selection process. Section 3.2 describes the role of the domain expert. Section 3.3 describes the three-source candidate discovery process. Section 3.4 describes the mention-count ranking method. Section 3.5 describes the filtering criteria applied to the candidate list. Section 3.6 presents the final study set of 28 packages. Section 3.7 describes the measurement procedure using the Domain X template. Section 3.8 describes the commonality analysis procedure. Section 3.9 describes the planned developer interview component.

3.1 Domain Selection

The first step of the Domain X methodology is to identify a suitable research software domain. To be applicable for the methodology, the chosen domain must meet four

criteria **?**: (i) it must have a well-defined and stable theoretical underpinning, ideally supported by standard textbooks; (ii) there must be a community of researchers and practitioners studying the domain; (iii) the software packages in the domain must include open-source options; and (iv) a preliminary search should suggest that there are close to 30 candidate software packages that qualify as research software.

The selected domain is *robotics software for motion planning and simulation*. Motion planning and simulation are established subfields of robotics with well-defined theoretical foundations **??**. There is an active international research community, regular conferences (such as ICRA, IROS, and RSS), and a substantial body of published work. A preliminary search confirmed the existence of numerous open-source software packages spanning simulation environments, physics engines, dynamics libraries, motion planning libraries, and robotics middleware.

The domain boundary is broader than "robotics simulators" alone. Many robotics workflows combine a simulator with planning libraries, physics engines, middleware, and supporting tools. Restricting the study to full simulators would make comparison simpler but would omit software that belongs to core motion planning and simulation workflows.

Earlier project discussions considered a narrower domain focused on inverse kinematics software. After consultation with the supervisor and domain expert, the scope was broadened to the current domain, which provides a richer set of candidate packages and better reflects the interconnected nature of robotics software ecosystems.

3.2 Interaction with the Domain Expert

The Domain X methodology emphasizes that a domain expert is an essential member of the assessment team. Pitfalls exist if non-experts attempt to produce an authoritative list of software or definitively rank packages without domain knowledge. Non-experts must rely on information available online, which has two known drawbacks: (i) online resources may contain false or inaccurate information; and (ii) online resources may omit relevant information that is so ingrained with experts that nobody thinks to record it explicitly **?**.

Domain experts may be recruited from academia or industry. The only requirements are knowledge of the domain and a willingness to be engaged in the assessment process. The domain expert does not need to be a software developer, but they should be a user of domain software. Given that domain experts are likely to be busy people, the methodology is designed to minimize the burden on their time **?**.

For this study, the domain expert is Dr. Matthew Giamou of the Department of Computing and Software at McMaster University. Dr. Giamou's research includes robotics, motion planning, and state estimation. He reviewed the candidate software list, confirmed that the scope was appropriate, and provided guidance on inclusion and exclusion decisions. Material in this thesis that has not yet received final expert review is marked as pending.

The domain expert was asked to review the candidate software list and provide feedback on scope and coverage. Dr. Giamou confirmed that the list was reasonable and that most of the shortlisted tools were familiar to him. He noted that several packages (ROS 2, MoveIt!, and OMPL) are not full simulators but can fit within a

coherent narrative if their roles are explained carefully. He also confirmed that excluding MATLAB/Simulink, Unity, NVIDIA Isaac Sim, and RaiSim on methodological grounds was acceptable and would not misrepresent the domain.

3.3 Candidate Software Discovery

Candidate software packages were identified from three types of sources: academic survey papers, community-curated software lists, and LLM-generated software lists. Using multiple source types reduces dependence on any single search method and provides broader coverage of the domain. The collected names were consolidated to merge equivalent entries (for example, "V-REP" and "CoppeliaSim" refer to the same package), and the consolidated list was used as input to the mention-count ranking described in Section 3.4.

3.3.1 Academic Sources

Academic sources identify software packages that appear in research discussions of robotics simulators, simulation tools, and related challenges. These sources included survey papers and comparative studies that discuss packages such as Gazebo, Webots, V-REP/CoppeliaSim, MuJoCo, and PyBullet in the context of robotics research. Academic sources connect software packages to published research contexts and provide a research-facing view of the domain.

Academic sources have known limitations as the sole basis for candidate discovery. Survey papers may emphasize packages relevant to a specific research question, robot type, or time period. Some active tools may not appear in the selected papers,

while older tools may remain visible because they were prominent when a survey was written. For these reasons, academic sources were combined with community-curated and LLM-assisted sources rather than used in isolation.

3.3.2 Community-Curated Sources

Community-curated sources broaden the view of robotics software practice beyond what appears in academic surveys. The source catalogue included GitHub topic pages, "awesome" lists (curated collections such as `awesome-robotics-libraries` and `awesome-robotics-simulation`), robotics software directories, best-of lists, and other community-maintained aggregations. Sources covered topics including motion planning, robotics simulation, robotics libraries, ROS 2 ecosystem tools, multibody dynamics simulation, and general robotic tooling.

These sources collect software that practitioners and community contributors consider relevant. They can reveal projects that are underrepresented in academic surveys and can show relationships between simulators, planning libraries, middleware, physics engines, and tools. Community-curated sources also vary in maintenance quality, inclusion criteria, and update frequency. They were therefore used as discovery aids rather than as indicators of software quality.

3.3.3 LLM-Assisted Sources

LLM-generated software lists were used as an auxiliary source for candidate discovery. This modifies the original Domain X methodology, which prescribes search engine queries and domain expert consultation as the primary discovery methods. The modification is recorded explicitly so that the methodology remains transparent

and reproducible in principle.

The reason for including LLM-assisted discovery is practical: large language models can surface software names that appear across a wide range of public information, functioning as a broad aggregation mechanism over existing public data. However, LLM output can also be incomplete, outdated, or incorrect. Names obtained from LLM-generated lists therefore require consolidation, filtering, and verification against public evidence before they can support any thesis claims. In this project, LLM-assisted sources contributed to candidate discovery but did not determine the final study set, and they were never treated as authoritative.

This approach was discussed with the supervisor, who noted that while the use of LLMs is technically a modification of the peer-reviewed methodology, it is defensible for two reasons: (i) LLMs were used together with academic and community-curated sources, not as the sole discovery method; and (ii) LLMs can be understood as a large-scale aggregation mechanism over existing public information, analogous in effect to a broad search over indexed web content.

3.4 Mention Count Ranking

After consolidating equivalent names across the collected sources, each unique software package was counted once per source to produce a mention count. A package that appeared in five academic papers and three community-curated lists would receive a mention count of eight. The consolidated ranking was recorded during the discovery phase and is a transparent input to the filtering process.

The top entries in the consolidated ranking include Gazebo (22 mentions), Webots (16), Bullet/PyBullet (16), CoppeliaSim/V-REP (13), MuJoCo (12), ODE (10),

ROS 2 (10), and OpenRAVE (9). These counts help organize the candidate list and make the selection process auditable. The full ranked list with mention counts is preserved in the project materials.

Mention count is a source-appearance signal and must be interpreted accordingly. It is not a measure of software quality, user adoption, research impact, maintainability, or suitability for any particular robotics task. A package can appear frequently because it is historically established, widely discussed in survey papers, broadly useful across application areas, or simply visible in the particular sources collected. A more specialized or newer package may be underrepresented even if it is technically strong. The mention count serves as one input to candidate filtering, not as a final evaluation or ranking of the software.

3.5 Filtering Criteria

The ranked candidate list was filtered to identify packages suitable for repository-based State-of-the-Practice measurement. In the Domain X methodology, the initial filters are scope, usage, and age, applied in priority order until the list reaches a manageable size of approximately 30 packages **?**. In this project, these filters are implemented through three criteria: open-source availability, repository-based measurability, and activity and maintenance status. Both the initial and filtered lists are preserved for traceability.

3.5.1 Open Source Availability

Open-source availability was required because the study depends on public software artifacts for measurement. The Domain X candidate criteria require that source code be viewable and express a preference for GitHub-style repositories from which empirical measures can be collected **?**. Source code, repository history, issue trackers, documentation, and release information provide the evidence base for the measurements reported in Chapter 5.

This criterion led to the exclusion of several prominent robotics tools whose core components are not open source:

- **MATLAB and Simulink** are major platforms in both robotics research and industry, widely used for algorithm development, control design, and simulation. However, MATLAB is proprietary software. Its source code is not publicly available, and its development process is not accessible through public repositories. These properties prevent the kind of repository-based measurement that this study depends on. Dr. Giamou confirmed that excluding MATLAB and Simulink does not misrepresent the domain.

- **Unity** is a widely used game engine with robotics simulation capabilities, but its core simulation engine is proprietary.

- **NVIDIA Isaac Sim** builds on NVIDIA Omniverse and provides advanced robotics simulation features, but its core platform components are not open source.

- **RaiSim** is a high-performance physics simulator for robotics, but its source code is not publicly available under an open-source license.

These exclusions reflect methodological constraints rather than judgements about the quality or importance of the excluded tools. Each of these platforms plays a significant role in robotics research and industry, and their exclusion limits the coverage of the study. This limitation is discussed further in Chapter 7.

3.5.2 Repository-Based Measurability

Repository-based measurability requires that a package have sufficient public evidence to support the measurement process described in Section 3.7. Relevant evidence includes source repositories, commit history, issue data, documentation, release information, build or installation instructions, test artifacts, and project metadata. Packages without sufficient public evidence cannot be measured consistently and were excluded.

This criterion also governs how multi-repository projects are handled. Some robotics software systems are distributed across multiple repositories or GitHub organizations. In such cases, the repositories most representative of the core software were selected for measurement, and this choice was documented as part of the measurement record. The same principle applies to projects that have changed names, migrated repositories, or spawned successor projects.

3.5.3 Activity and Maintenance Status

Activity and maintenance status were considered because the study focuses on contemporary software practice. In the Domain X methodology, age is one of the initial filtering criteria, measured by the last date when a change was made, with exceptions for older packages that remain highly recommended and currently in use **?**. Tools

that are no longer maintained provide weaker evidence about current development practices.

The primary activity indicator used in this study is the Domain X alive/dead rule: a package is considered alive if it has received commits or version releases within the last 18 months from the measurement date. The following packages were excluded on maintenance grounds:

- **MRDS** (Microsoft Robotics Developer Studio) has not received updates since 2012 and is no longer supported by Microsoft.
- **USARSim** (Unified System for Automation and Robot Simulation) is no longer actively maintained and does not represent current practice.

Activity-related evidence is a snapshot. Repository activity, last commit dates, and release status can change after measurement. The measurement records in Chapter 5 include the measurement date to make the snapshot nature of the data explicit.

3.6 Final Software Set

After ranking by mention count and applying the filtering criteria described above, the final study set consists of 28 software packages. Table 3.1 lists the packages alphabetically with their primary roles in the robotics software ecosystem.

Table 3.1: Final software set with primary roles.

Software	**Primary Role**
AirSim	Simulator (aerial vehicles)
Bullet / PyBullet	Physics engine
CARLA	Simulator (autonomous driving)
CoppeliaSim (V-REP)	Simulator (general robotics)
DART	Dynamics library
Drake	Toolbox (dynamics, planning, control)
Flightmare	Simulator (quadrotor)
Gazebo	Simulator (general robotics)
GraspIt!	Manipulation (grasping)
Habitat	Simulator (embodied AI)
MORSE	Simulator (academic robotics)
MoveIt!	Manipulation (motion planning)
MRPT	Library (mobile robotics)
MuJoCo	Physics engine
Newton Dynamics	Physics engine
ODE (Open Dynamics Engine)	Physics engine
OMPL	Motion planning library
OpenRAVE	Planning framework
OpenRTM-aist	Middleware (RT components)
OROCOS	Control framework
PhysX (open-source SDK)	Physics engine
Pinocchio	Dynamics library
Project Chrono	Multi-physics simulation
ROS 2	Middleware and SDK
Simbody	Multibody dynamics library
SOFA	Simulation framework

This set is deliberately heterogeneous. Most entries are associated with simulation, physics, dynamics, planning, or robotics infrastructure, but they are not direct substitutes for one another. ROS 2, MoveIt!, and OMPL are boundary cases because they are not full simulators in the same sense as Gazebo or Webots. They are retained because the study scope is robotics software for motion planning and simulation, not simulator applications alone. Chapter 4 provides a detailed analysis of software categories, commonalities, and variabilities across this set.

3.7 Measurement Procedure

The measurement procedure follows the Domain X method for collecting structured evidence about software packages in a selected domain **?**. For each of the 28 selected packages, the study gathered source code, documentation, publications where applicable, repository metadata, issue-tracker evidence, release information, and installation evidence. The measurement record identifies the specific repository or repositories measured, since some robotics projects are distributed across multiple repositories.

3.7.1 Measurement Template

The primary measurement instrument is the Domain X Appendix B measurement template. The template contains 108 questions grouped into the following sections:

1. **Summary Information** (17 questions): software name, URL, affiliation, purpose, number of developers, funding, release dates, status, license, platforms, software category, development model, publications, source code URL, programming languages, evidence of performance consideration, and additional

comments.

2. **Installability** (15 questions): installation instructions (presence, organization, linearity, completeness), compatible operating systems, automation, error handling, validation, number of steps, operating system used, dependencies, and uninstall behaviour.

3. **Correctness and Verifiability** (9 questions): requirements or theory documentation, correctness techniques, tutorial availability and quality, unit tests, and continuous integration evidence.

4. **Surface Reliability** (5 questions): installation and tutorial breakage, error messages, and recoverability.

5. **Surface Robustness** (2 questions): handling of unexpected input and edge cases.

6. **Surface Usability** (5 questions): tutorial, user manual, user characteristics, support model.

7. **Maintainability** (8 questions): version number, contribution guidance, available artifacts, issue tracking, issue closure rate, comment percentage, and version control.

8. **Reusability** (2 questions): number of code files and API documentation.

9. **Surface Understandability** (9 questions): code formatting, coding standards, identifier quality, constant usage, comment clarity, algorithm references, parameter ordering, and modularization.

10. **Visibility and Transparency** (4 questions): development process definition, process documentation, development environment documentation, and release notes.

Applying the same template to every package ensures that comparisons are based on a consistent set of observations rather than ad hoc impressions. Each quality section concludes with an overall impression score on a scale of 1 to 10, assigned using the scoring guidelines in the Domain X Appendix C impression calculator.

3.7.2 Repository Metrics

Repository metrics are collected separately from the quality impression scores. Three categories of repository data are gathered:

1. **GitHub-style metrics:** stars, forks, watchers, open pull requests, closed pull requests, number of developers, open issues, closed issues, initial release date, and last commit date.

2. **GitStats-style metrics:** number of text-based files, number of binary files, total lines, lines added, lines deleted, total commits, commits by year (last 5 years), and commits by month (last 12 months).

3. **SCC-style metrics:** number of text-based files, total lines, code lines, comment lines, and blank lines.

From these raw data, three processed measures are calculated:

1. **Alive/dead status:** alive is defined as the presence of commits or version releases within the last 18 months from the measurement date.

2. **Percentage of identified issues that are closed:** computed as closed issues divided by total identified issues.

3. **Percentage of code that is comments:** computed as comment lines divided by total lines.

3.7.3 Measurement Execution

The Domain X methodology recommends performing installation-based measurements in a controlled environment, typically a virtual machine, to ensure a clean and reproducible starting state **?**. For this study, installation measurements were performed on a Linux-based virtual machine. The operating system, installation steps, dependencies, and any issues encountered were recorded for each package.

The methodology also recommends a time budget to keep the overall measurement workload feasible. The target is approximately 160 person-hours for the entire domain, with 1 to 4 hours allocated per package for template completion and up to 2 hours for installation **?**. Measurement was conducted during May 2026. All time-sensitive data (last commit dates, issue counts, stars, forks, etc.) should be understood as snapshots taken during this period.

Several measurement questions require explicit interpretation rules:

- **Last commit date** is recorded at the time of measurement and may change for active repositories.

- **Percentage of identified issues that are closed** is computed consistently as closed issues divided by total identified issues, using GitHub issue data where available.

- **Evidence that performance was considered** is assessed by searching software artifacts for explicit references to speed, storage, throughput, performance optimization, parallelism, multi-core processing, or similar considerations.
- **Percentage of code that is comments** is computed using the SCC tool's line-based method, as comment lines divided by total lines.

3.7.4 Ranking with AHP

After the quality measures are collected, the Domain X methodology uses the Analytic Hierarchy Process (AHP) to support ranking across the nine quality categories **?**. AHP is a decision-making technique that uses pairwise comparisons between alternatives to calculate relative scores. It organizes multiple criteria in a hierarchical structure and enables the combination of individual quality rankings into an overall ranking based on the relative priorities assigned to each quality.

In this study, AHP is applied to the nine quality impression scores for the 28 software packages. The AHP process produces a ranking that is an analysis aid for comparing evidence under the chosen criteria. It is not a universal claim that one package is best for every robotics use case. Sensitivity analysis can be applied to assess whether the ranking is stable under reasonable variations in quality weightings. Any AHP ranking reported in this thesis appears only after the underlying measurements are available and verified.

3.8 Commonality Analysis Procedure

The commonality analysis identifies shared and variable capabilities across the selected software packages, treating them as members of a software family as defined by Parnas **?** and Weiss **?**. The analysis supplements repository-based metrics by describing what the packages do and how their roles differ, providing context for interpreting the measurement results reported in Chapter 5.

Following the Domain X methodology, the analysis proceeds in six steps:

1. **Write an introduction** to the domain and the software family.

2. **Write an overview of domain knowledge,** summarizing the key concepts and terminology relevant to robotics software for motion planning and simulation.

3. **List commonalities:** identify goals, assumptions, functions, artifacts, and domain concepts that recur across multiple family members. For example, many packages in the study set share the goal of simulating robot dynamics, the assumption that robot models are described using standard formats (such as URDF or SDF), and the function of providing an API for programmatic control.

4. **List variabilities:** identify dimensions on which family members differ. These include software role (simulator, physics engine, planning library, middleware), supported platforms, licensing, programming language interfaces, documentation style, and intended use cases.

5. **List parameters of variation:** for each variability, describe the possible values. For example, the software role parameter can take values such as general-purpose simulator, domain-specific simulator, physics engine, dynamics library, motion planning library, manipulation framework, or middleware.

6. **Add terminology, definitions, and acronyms** that are standard in the domain, to ensure that the analysis is accessible to readers outside robotics.

The procedure first groups the selected packages into broad categories based on their primary roles: robotics simulators, physics and dynamics engines, and planning or middleware tools. It then identifies capabilities that appear across multiple categories, such as support for robot model representation, simulated environments, motion planning workflows, dynamics computation, and integration with robotics ecosystems.

Because the selected set is heterogeneous, the commonality analysis does not force all tools into a single comparison template. Its purpose is to describe shared concerns and meaningful differences, not to claim that every package provides identical functionality. The analysis is based on checked evidence from documentation, repositories, publications, and other reliable project materials. Capabilities that appear likely but have not been verified remain marked as needing confirmation.

3.9 Interview Component

The Domain X methodology includes a developer survey component designed to capture information that is not visible in public repositories: the development process as experienced by practitioners, the pain points they encounter, their perceptions of

software quality threats, and the strategies they use to address those threats **?**. The methodology prescribes semi-structured interviews based on a list of 20 questions covering developer background, project organization, user understanding, development obstacles (past and present), documentation practices, and specific software qualities including maintainability, understandability, usability, and reproducibility.

For this study, the interview component is associated with research ethics approval through the McMaster Research Ethics Board (MacREM). The project materials indicate that interviews with software developers, maintainers, or programmers associated with the studied packages may form part of the broader State-of-the-Practice exercise.

As of the current thesis draft, approved interview data (including transcripts, participant responses, approved summaries, or final interview findings) is not yet available. The interview component is therefore described as planned or pending. Chapter 6 is reserved for this material and currently describes the planned protocol, recruitment strategy, and analysis framework. Interview findings will be added only when supported by approved data.

The methodology recommends sending interview requests to developers from all packages in the study set to avoid selection bias, with an expected response rate between 15% and 30% based on prior applications of the methodology **?**. Interview contacts are typically identified through project websites, code repositories, publications, or institutional biography pages.

Chapter 4

Selected Software and Commonality Analysis

This chapter presents the 28 software packages selected for the study and analyzes them as members of a software family. Section 4.1 provides an overview of the selected set. Section 2.2 groups the packages into three broad categories based on their primary roles. Section 4.3 identifies common capabilities shared across the family. Section 4.4 identifies dimensions on which the packages differ. Section 4.5 describes the parameters of variation and their binding times. Section 4.6 discusses boundary cases that occupy intermediate positions in the classification.

The commonality analysis follows the approach described by Weiss **?** and is based on evidence from project documentation, repositories, publications, and other reliable sources. Capabilities described as likely but not yet verified against project materials are explicitly noted.

4.1 Overview of Selected Software

The 28 software packages selected through the discovery and filtering process described in Chapter 3 represent a cross-section of the robotics software ecosystem for motion planning and simulation. The set includes tools developed in academic research groups (such as Drake, DART, and OMPL), by research and industry consortia (such as ROS 2 and Gazebo), by corporate research labs (such as MuJoCo, AirSim, and Habitat), and by individual open-source communities (such as ODE and Newton Dynamics).

The packages span a period of active development from 2001 (ODE's initial release) to the present, with most showing recent repository activity as of the May 2026 measurement date. All 28 packages are publicly available under open-source licenses, with Apache 2.0, BSD, MIT, and LGPL being the most common. The majority of packages are written primarily in C++ with Python bindings, reflecting the dominant languages in the robotics software community. All 28 packages support Linux, and most also support Windows and macOS.

This chapter does not report measurement scores or quality rankings; those appear in Chapter 5. Its purpose is to describe what the selected packages do, how they relate to one another, and what dimensions of variation characterize the software family, so that the measurement results in Chapter 5 can be interpreted in context.

4.2 Software Categories

The selected packages can be grouped into three broad categories based on their primary function within a robotics workflow. These categories are not strict boundaries (some packages span multiple roles) but they provide a useful structure for the commonality analysis.

4.2.1 Robotics Simulators

Robotics simulators provide virtual environments for representing robots, worlds, sensors, and actuators. They enable researchers and engineers to test algorithms, validate designs, and collect synthetic data before deploying to physical hardware. Packages in this category typically offer graphical rendering, physics simulation, sensor models, and programmatic interfaces for controlling simulated robots.

The following packages from the study set are primarily classified as robotics simulators:

- **Gazebo** (including the Ignition/Gazebo Sim lineage) is a long-established open-source 3D robotics simulator developed by Open Robotics. It provides high-fidelity physics (using multiple physics engine backends including ODE, Bullet, DART, and Simbody), rendering, and sensor models, with deep integration into the ROS ecosystem.

- **Webots** is a multi-platform robot simulator originally developed at EPFL and now maintained by Cyberbotics. It supports modeling, programming, and simulating robots, vehicles, and mechanical systems, with a large library of pre-built robot models.

- **CoppeliaSim** (formerly V-REP) is a versatile robot simulation framework developed by Coppelia Robotics. It supports rapid algorithm development, factory automation simulation, prototyping, verification, and digital twin applications.
- **AirSim** is an open-source simulator for autonomous vehicles (drones and ground vehicles) built on Unreal Engine, originally developed by Microsoft Research. It provides photorealistic environments and sensor models for AI and reinforcement learning research.
- **CARLA** is an open-source autonomous driving simulator built on Unreal Engine, developed by the Computer Vision Center in Barcelona with Intel and community support. It provides photorealistic urban environments and a comprehensive sensor suite for autonomous driving research.
- **MORSE** (Modular OpenRobots Simulation Engine) is a generic academic robotic simulator based on Blender and the Bullet physics engine, emphasizing middleware-agnostic design compatible with ROS, YARP, and other frameworks.
- **Habitat** is a high-performance 3D simulator for embodied AI research developed by Meta AI Research. It enables training and evaluation of embodied AI agents in photorealistic indoor environments.
- **Flightmare** is a flexible modular quadrotor simulator developed by the Robotics and Perception Group at the University of Zurich and ETH Zurich. It separates rendering from the RL environment for flexible configuration between fast and photorealistic modes.

Several additional packages provide simulation capabilities as part of broader functionality. Drake includes simulation features within its model-based design toolbox. MRPT includes a 3D simulation environment alongside its mobile robotics libraries. SOFA provides interactive simulation for medical and soft robotics applications. These packages may be considered simulators in some contexts but serve broader roles.

4.2.2 Physics Engines and Dynamics Libraries

Physics engines and dynamics libraries provide the computational foundation for simulating rigid-body and multibody dynamics, collision detection, and contact resolution. They are often used as backends by higher-level simulators but can also be used directly by researchers and developers.

The following packages are primarily classified as physics engines or dynamics libraries:

- **Bullet / PyBullet** is a widely used open-source physics engine for simulation, robotics, and deep reinforcement learning. PyBullet provides Python bindings that have made it popular in the machine learning community.

- **MuJoCo** (Multi-Joint dynamics with Contact) is a general-purpose physics simulator developed originally at the University of Washington and now maintained by Google DeepMind. It is designed for fast and accurate simulation of articulated structures and is widely used in robotics and reinforcement learning research.

- **ODE** (Open Dynamics Engine) is one of the earliest open-source libraries for

simulating rigid body dynamics. It has been used extensively in robotics and served as a physics backend for Gazebo Classic.

- **PhysX** is NVIDIA's high-performance real-time physics engine SDK. The open-source SDK provides capabilities for simulating rigid bodies, particles, vehicles, and deformable bodies, primarily targeted at real-time simulation applications.
- **DART** (Dynamic Animation and Robotics Toolkit) is a C++ physics engine and kinematics/dynamics library developed at Georgia Tech and other institutions. It emphasizes accurate and stable simulation for manipulation and locomotion research.
- **Simbody** is a high-performance multibody dynamics library developed at Stanford University for simulating articulated biomechanical and mechanical systems. It is the physics engine underlying OpenSim (biomedical simulation software).
- **Pinocchio** is a fast and flexible C++ library for rigid-body dynamics computations and their analytical derivatives, developed primarily at Inria and LAAS-CNRS. It implements state-of-the-art algorithms for kinematics, dynamics, and Jacobians.
- **Project Chrono** is a high-performance open-source multi-physics simulation library developed at the University of Wisconsin-Madison and the University of Parma. It supports multibody dynamics, finite element analysis, vehicle dynamics, and GPU-accelerated simulation.
- **Newton Dynamics** is a high-performance physics simulation engine optimized

for real-time deterministic simulation with exact arithmetic. It is used in game development and robotics simulation.

- **SOFA** (Simulation Open Framework Architecture) is an open-source real-time simulation framework developed primarily at Inria. It focuses on deformable object simulation, finite element methods, and haptic feedback, with applications in surgical simulation and soft robotics.

4.2.3 Motion Planning and Robotics Middleware

Motion planning libraries and robotics middleware support planning, integration, communication, and robotic system development. These packages are not simulators, but they belong to the robotics workflows that combine planning, simulation, and execution.

The following packages are primarily classified as planning or middleware tools:

- **ROS 2** (Robot Operating System 2) is an open-source software development kit and middleware framework for building robot applications. It provides communication infrastructure (based on DDS), hardware abstraction, drivers, libraries, visualization tools, and a large ecosystem of community packages. ROS 2 is the successor to ROS 1 and represents a major redesign for industrial and production use.

- **MoveIt!** (specifically MoveIt 2 for ROS 2) is an open-source robotics manipulation framework led by PickNik Robotics. It provides motion planning, kinematics, collision checking, trajectory execution, and perception integration for robot arms and manipulators, integrating OMPL and other planners as

plugins.

- **OMPL** (Open Motion Planning Library) is an open-source C++ library for sampling-based motion planning algorithms developed at Rice University. It provides implementations of widely used planners such as PRM, RRT, RRT*, and EST, and is the default planning backend for MoveIt!.

- **OpenRAVE** (Open Robotics Automation Virtual Environment) is an open-source robotics planning framework developed at Carnegie Mellon University. It focuses on simulation and analysis of kinematic and geometric information for motion planning, particularly for manipulation tasks.

- **Drake** is an open-source C++ and Python toolbox for model-based design and verification of robotics systems, developed at MIT and the Toyota Research Institute. It covers multibody dynamics, trajectory optimization, model predictive control, perception, and manipulation planning.

- **OROCOS** (Open RObot COntrol Software) is an open-source C++ framework for real-time robot control developed at KU Leuven. Its major components include the Real-Time Toolkit for component-based hard real-time software and the Kinematics and Dynamics Library.

- **MRPT** (Mobile Robot Programming Toolkit) is an open-source cross-platform C++ library for mobile robotics research, providing algorithms for SLAM, computer vision, motion planning, and sensor data processing.

- **YARP** (Yet Another Robot Platform) is open-source middleware for humanoid

and general robotics, developed at the Istituto Italiano di Tecnologia. It provides communication infrastructure for distributed robot systems and is the primary middleware for the iCub humanoid robot platform.

- **OpenRTM-aist** is an implementation of the OMG-standardized Robot Technology Component standard, developed at AIST in Japan. It provides a component-based framework for creating and connecting robot software modules.

- **GraspIt!** is a simulation environment for robotic grasping research developed at Columbia University. It supports loading robot hand models and 3D object models to plan, evaluate, and visualize grasps.

4.3 Commonalities

Despite the diversity of the selected packages, several capabilities and concerns recur across the software family. These commonalities are organized around the abstraction of a typical robotics software workflow: model representation, computation, interaction, and documentation.

C1: Robot Model Representation. All packages in the study set provide or consume some form of robot model representation. Simulators and physics engines represent robot kinematics and dynamics through internal model structures (often supporting standard formats such as URDF, SDF, or MJCF). Motion planning libraries operate on kinematic and geometric descriptions of robots and environments. Middleware packages define standardized message formats for communicating robot state.

C2: Physics or Kinematics Computation. The packages in the simulator

and physics engine categories share the capability of computing robot motion under physical constraints. This includes forward and inverse kinematics, rigid-body dynamics, collision detection, and contact resolution. Even middleware and planning tools typically include kinematic computation capabilities or depend on libraries that provide them.

C3: Programmatic Interface. All 28 packages provide a programmatic interface (typically a C++ API, often with Python bindings) that allows developers to control the software, configure simulations or planners, and integrate the package into larger robotics workflows.

C4: Open-Source Repository Hosting. All 28 packages use public version control hosting (GitHub for 27 of 28 packages) with visible commit history, issue tracking, and pull request management. This shared infrastructure is both a selection criterion and a commonality of the study set.

C5: Documentation. All packages provide some form of user-facing documentation, typically including installation instructions, a getting-started tutorial, and API reference material. The depth, currency, and completeness of this documentation vary considerably, as discussed in Chapter 5.

C6: Integration with Robotics Ecosystems. Most packages provide or support integration with broader robotics ecosystems. ROS 2 integration is the most common pattern, with many simulators offering ROS 2 interfaces for publishing sensor data and accepting control commands. Some packages (such as Gazebo and MoveIt!) are deeply embedded in the ROS ecosystem, while others maintain their own integration frameworks.

C7: Simulation or Planning Loop. Simulators and planners share a common

architectural pattern: a loop that iteratively updates the system state. In simulators, this is the physics stepping loop that advances the simulation in time. In planners, it is the search or optimization loop that explores the configuration space to find a valid path.

4.4 Variabilities

The selected packages differ along several important dimensions. These variabilities are not deficiencies; they reflect the different roles that packages play within the robotics software ecosystem.

V1: Primary Software Role. The most fundamental variability is the role a package plays in a robotics workflow. As described in Section 2.2, packages range from full simulation environments (Gazebo, Webots, CoppeliaSim) to physics engines (Bullet, MuJoCo, ODE) to planning libraries (OMPL) to middleware frameworks (ROS 2, YARP).

V2: User Interface Model. Packages differ in how users interact with them. Some provide graphical user interfaces with 3D rendering, scene editors, and interactive controls (Gazebo, Webots, CoppeliaSim). Others are primarily libraries or APIs intended for programmatic use (OMPL, Pinocchio, ODE). Some provide both (Drake, MRPT).

V3: Domain Specificity. Some packages are general-purpose robotics simulators or engines (Gazebo, Bullet, MuJoCo), while others target specific application domains: CARLA and AirSim for autonomous vehicles, Flightmare for quadrotor flight, Habitat for embodied AI in indoor environments, SOFA for medical simulation, and GraspIt! for robotic grasping.

V4: Physics Fidelity and Approach. Physics engines differ in their simulation approach and fidelity. Some prioritize speed for real-time or reinforcement learning applications (MuJoCo, Bullet), while others emphasize physical accuracy for engineering analysis (Project Chrono, Simbody). Differences include the choice of constraint solver, collision detection algorithm, integration method, and support for deformable bodies or fluid dynamics.

V5: Platform Support. While all packages support Linux, support for other platforms varies. Some packages provide first-class support for Windows and macOS (Webots, CoppeliaSim, Bullet), while others are primarily developed for Linux with limited cross-platform support.

V6: Programming Language Interfaces. C++ is the dominant implementation language across the study set, but packages differ in the range and quality of language bindings they provide. Python bindings are increasingly common because Python has become central to robotics research and machine learning. Fewer packages provide interfaces for languages such as MATLAB, Java, or Rust.

V7: ROS Integration Depth. Packages vary in their relationship to the ROS ecosystem. Some (Gazebo, MoveIt!) are tightly integrated and depend on ROS for core functionality. Others (Webots, CARLA) provide ROS bridges as optional components. Some (YARP, OROCOS) are alternatives to ROS that provide their own middleware infrastructure.

V8: License Model. Although all packages are open source, they use different licenses. The most common are Apache 2.0, BSD (2-clause and 3-clause), MIT, and LGPL. License choice affects how the software can be reused and integrated into other projects.

V9: Community and Governance. Packages differ in their development governance. Some are stewarded by established organizations (Open Robotics for Gazebo and ROS 2, Google DeepMind for MuJoCo, NVIDIA for PhysX), some by academic research groups (Drake at MIT/TRI, OMPL at Rice, DART at Georgia Tech), and some by individual maintainers or small communities (ODE, Newton Dynamics, GraspIt!).

4.5 Parameters of Variation

For each variability identified in Section 4.4, Table 4.1 summarizes the parameters of variation and their typical binding times. Binding time indicates when a particular choice is fixed: at specification time (when the software is designed), build time (when it is compiled), configuration time (when it is set up), or run time (during execution).

Table 4.1: Parameters of variation and binding times.

Variability	**Parameters of Variation**	**Binding Time**
V1: Primary role	Simulator, physics engine, dynamics library, motion planning library, manipulation framework, middleware, multi-purpose toolbox	Specification time
V2: User interface	Graphical (3D scene, editor), programmatic (API only), both	Specification time
V3: Domain specificity	General-purpose robotics, autonomous driving, aerial vehicles, embodied AI, medical/soft robotics, grasping, mobile robotics	Specification time
V4: Physics fidelity	Real-time (game-quality), high-fidelity (engineering-quality), configurable trade-off	Specification / configuration time
V5: Platform support	Linux-only, Linux + Windows + macOS, web-based	Specification / build time
V6: Language bindings	C++ only, C++ with Python, multi-language (Python, MATLAB, Java, etc.)	Build / configuration time
V7: ROS integration	Native (ROS-dependent), bridge (optional ROS interface), independent (separate middleware), alternative (non-ROS middleware)	Configuration time
V8: License	Permissive (BSD, MIT, Apache 2.0), reciprocal (LGPL, GPL)	Specification time
V9: Governance	Single institution or company,	Specification

The binding times reflect when a user or integrator can make choices. A package's primary role and governance model are fixed at specification time—they are inherent to the software's design. Platform support is largely determined at build time, though some packages allow configuration-time choices. ROS integration depth can often be chosen at configuration time, as many packages offer optional ROS bridges. Physics fidelity involves trade-offs that may be configured at both specification time (choice of engine) and run time (choice of solver parameters or time step).

4.6 Boundary Cases

Several packages in the study set occupy intermediate positions in the classification presented above. These boundary cases are not classification errors; they illustrate the interconnected nature of the robotics software ecosystem and the difficulty of drawing sharp boundaries between categories.

ROS 2 is classified as middleware in this analysis, but its scope extends well beyond communication infrastructure. ROS 2 provides an SDK with libraries, drivers, visualization tools (RViz), simulation integration (through Gazebo), and a package management system. It is the de facto standard for integrating robotics software components and serves as the foundation on which many other packages in the study set operate. Its inclusion in a study of motion planning and simulation software is justified by its central role in the workflows that combine these capabilities.

Drake spans multiple categories. It provides simulation capabilities without being a full simulator in the sense of Gazebo or Webots. It includes sophisticated multibody dynamics without being solely a physics engine. It offers trajectory optimization and model predictive control without being only a planning library. Drake is best

understood as a model-based design toolbox that integrates dynamics, planning, and control into a unified framework.

MoveIt! and **OMPL** are closely related but serve different roles. OMPL is a general-purpose motion planning library that can be used independently. MoveIt! is a manipulation framework that integrates OMPL (among other planners) with kinematics, collision checking, and trajectory execution for robot arms. OMPL is classified as a motion planning library and MoveIt! as a manipulation framework, but the boundary between them reflects architectural layering rather than functional separation.

SOFA is classified as a simulation framework but differs from general-purpose robotics simulators in its focus on deformable objects and finite element methods. Its primary application domain is surgical simulation and medical education, making it a domain-specific simulator rather than a general robotics tool.

These boundary cases are not treated as measurement anomalies. In the chapters that follow, measurements are reported for each package individually, and comparisons are interpreted in the context of each package's role within the robotics software ecosystem.

Chapter 5

Measurement Results

This chapter reports the measurement results for the 28 selected software packages. All data was collected during May 2026 and represents a snapshot of repository state, documentation, and project activity at that time. The measurements follow the Domain X Appendix B template and cover the nine quality categories described in Section 2.4. Section 5.1 presents summary information and project metadata. Sections 5.2 through 5.10 report results for each quality category. Section 5.11 reports repository metrics. Section 5.12 synthesizes the findings.

5.1 Repository and Project Metadata

Table 5.1 presents summary metadata for all 28 packages, including the primary development organization, license, initial release date, platform support, and primary programming languages. The packages are listed alphabetically.

Table 5.1: Summary metadata for the 28 selected software packages (May 2026).

Software	**Primary Affiliation**	**License**	**Initial Release**	**Primary Languages**
AirSim	Microsoft Research	MIT	2017	C++, Python
Bullet/PyBullet	Community	zlib	2006	C++, Python
CARLA	CVC Barcelona, Intel	MIT	2017	C++, Python
CoppeliaSim	Coppelia Robotics	GPL + comm.	2019	C++, Python
DART	Georgia Tech	BSD	2011	C++, Python
Drake	TRI, MIT	BSD	2010	C++, Python
Flightmare	U. Zurich, ETH	MIT	2020	C++, Python
Gazebo	Open Robotics	Apache-2.0	2014	C++, Python
GraspIt!	Columbia U.	GPL v3+	2010	C++, Python
Habitat	Meta AI Research	MIT	2019	C++, Python
MORSE	LAAS-CNRS	BSD	2009	C, Python
MoveIt!	PickNik Robotics	BSD	2011	C++, Python
MRPT	U. Almería	BSD	2010	C++, Python

All 28 packages are public and open source, though one (CoppeliaSim) uses a mixed licensing model combining GPL with commercial and educational licenses. The most common licenses are BSD (11 packages), Apache 2.0 (5), and MIT (4). The packages span more than two decades of development history: ODE is the oldest, with its initial release in 2001, while PhysX's open-source SDK was released in 2022.

C++ is the dominant implementation language, used as a primary language by all 28 packages. Python is a primary or secondary language in 26 of 28 packages because of its growing role in robotics research and its use for scripting, bindings, and machine learning integration. All 28 packages support Linux, 27 support Windows, and 26 support macOS.

As of the May 2026 measurement date, 23 of the 28 packages (82%) were classified as alive under the Domain X 18-month activity rule. Five packages were classified as dead: ODE (last commit April 2026 but with very low activity; status borderline), OpenRAVE (last commit August 2024), MORSE (last commit March 2022), GraspIt! (last commit July 2020), and Flightmare (last commit May 2023).

The number of developers—defined as anyone who has made at least one accepted commit—ranges from 6 (CoppeliaSim, Flightmare) to 598 (MoveIt!). The median is 127. The total number of commits across all packages exceeds 230,000. Drake leads with 34,786 commits, followed by SOFA (27,404) and Project Chrono (19,510). These repository-level metrics are snapshots and should not be interpreted as direct measures of project scale or quality without considering factors such as repository age, commit granularity, and whether development activity is concentrated in a single repository.

5.2 Installability

Installability measures the effort required to install and uninstall software in a specified environment. All installation measurements were performed on Linux-based virtual machines. Table 5.2 summarizes key installability indicators across the study set.

Table 5.2: Key installability indicators across 28 packages.

Indicator	**Packages**	**%**
Installation instructions present	28	100%
Instructions in a single document or page	26	93%
Instructions are linear (single series of steps)	26	93%
Instructions assume no pre-installed dependencies	22	79%
Compatible OS versions listed	17	61%
Installation automation provided (script, makefile, etc.)	28	100%
Installation validation procedure available	27	96%
Dependency installation instructions provided	24	86%
Required package versions listed	19	68%
No problems caused by uninstall (where available)	27	96%

All 28 packages provide installation instructions, and all provide some form of installation automation (makefiles, scripts, package managers, or container definitions). This is a strong result compared with prior State-of-the-Practice studies in other domains. For context, the Medical Imaging study **?** reported that only about half of MI packages were successfully installed.

The number of installation steps ranged from 1 to 11, with a median of 2. The number of extra software packages required before or during installation ranged from 0 to 10, with a median of 1. GraspIt! required the most manual effort (11 steps, 10 extra packages), consistent with its dead status and lack of recent maintenance. At the other extreme, several packages (Webots, MuJoCo, DART, Simbody, and others) required no extra packages and could be installed in a single step using package managers or pre-built binaries.

Installation broke for only 2 of the 28 packages during measurement (OpenRAVE and Newton Dynamics); in both cases the installation instance was recoverable. No package broke during initial tutorial testing.

The overall installability impression scores (on a 1–10 scale) range from 4 to 10, with a mean of 7.4. The top-scoring packages are Gazebo (10), Pinocchio (10), Bullet/PyBullet (9), MORSE (9), and OMPL (9). The lowest-scoring packages are ODE (4), PhysX (4), and GraspIt! (5). For ODE, the low score reflects outdated installation documentation (the official guide dates from 2006 and refers to GNU make, VC6 project files, and manual copying of library files). For PhysX, the low score reflects difficulty locating the correct build documentation.

5.3 Correctness and Verifiability

Correctness and verifiability assess whether a program matches its specification and the ease with which correctness can be checked. Table 5.3 summarizes key indicators.

Table 5.3: Correctness and verifiability indicators.

Indicator	**Packages**	**%**
Reference to requirements specifications or theory manuals	26	93%
Getting-started tutorial present	28	100%
Tutorial instructions are linear	27	96%
Tutorial provides expected output	26	93%
Tutorial output matches expected output	24	86%
Unit tests available	27	96%
Evidence that performance was considered	28	100%

All 28 packages use at least one technique to build confidence in correctness. The most common technique is automated testing, used by all packages. Assertions in code are used by 24 of 28 packages (86%). Documentation generation tools (Doxygen, Sphinx, Javadoc) are used by 12 packages (43%). Several packages combine multiple techniques: Drake, DART, Pinocchio, OMPL, Project Chrono, and Habitat use automated testing, assertions, and documentation generation together.

All 28 packages provide a getting-started tutorial, and 27 of 28 provide linear tutorial instructions. Twenty-six packages provide an expected tutorial output, and in 24 cases the measured output matched the expected output. The four cases where output did not match or was not applicable correspond to packages where the tutorial required environment-specific configuration not captured by the standard measurement procedure.

Unit tests are available for 27 of 28 packages. CoppeliaSim's unit test status is unclear because its test infrastructure is not publicly visible in the same way as the

other packages, a consequence of its mixed commercial/open-source licensing model.

The overall correctness and verifiability impression scores range from 5 to 10, with a mean of 7.7. The top-scoring packages are Gazebo (10), Drake (10), ROS 2 (9), OMPL (9), Pinocchio (9), and MoveIt! (9). The lowest-scoring are GraspIt! (5) and Newton Dynamics (5).

5.4 Surface Reliability

Surface reliability assesses failure-free operation during the limited interactions performed during measurement: installation and initial tutorial use. These are surface checks, not comprehensive reliability experiments.

As noted in Section 5.2, installation broke for 2 of 28 packages (OpenRAVE and Newton Dynamics). In both cases, a descriptive error message was displayed and the installation instance was recoverable. No package broke during initial tutorial testing.

The overall surface reliability impression scores range from 6 to 10, with a mean of 7.7. The top-scoring packages are Gazebo (10), Drake (10), ROS 2 (9), OMPL (9), Pinocchio (9), and MoveIt! (9). Only two packages scored below 7: Newton Dynamics (6) and GraspIt! (6).

The generally high surface reliability scores reflect the fact that most packages installed without incident and all tutorials ran without breakage. This is a positive signal for the domain, though it should be interpreted cautiously: the measurement procedure involves a shallow interaction (installation plus one tutorial) on a single operating system, and does not constitute a systematic reliability evaluation.

5.5 Surface Robustness

Surface robustness assesses whether software handles unexpected or unanticipated input reasonably, including edge cases such as malformed input files. All 28 packages were assessed as handling unexpected input reasonably (producing appropriate error messages or graceful degradation rather than crashes or undefined behaviour). Twenty-seven of 28 packages were assessed as handling newline conversion (replacing newlines with carriage returns and newlines in plain-text input files) gracefully.

The overall surface robustness impression scores range from 5 to 10, with a mean of 7.3. Gazebo scores highest (10), followed by Drake (9). The lowest-scoring package is GraspIt! (5). The narrow range of scores reflects the binary nature of the surface robustness checks in the Domain X template: most packages either pass both checks or pass one of two, resulting in similar scores.

5.6 Surface Usability

Surface usability assesses the presence and quality of user-facing documentation, tutorials, and support infrastructure. It is not a full usability evaluation; no user studies or task-completion experiments were conducted.

All 28 packages provide a getting-started tutorial and a user manual. This is a notably strong result. In contrast, the Medical Imaging study **?** found that only 22 of 29 MI packages (76%) provided a user manual. Only one package in the current study (MuJoCo) documents expected user characteristics, which is consistent with findings from other domains where this practice is rare **??**.

Table 5.4 summarizes the user support models adopted by the study packages.

Most packages use multiple support channels.

Table 5.4: User support models across 28 packages.

Support Channel	**Packages Using**
GitHub Issues	28
GitHub Discussions	10
Official forum or discussion board	9
Mailing list / Google Group	8
Discord server	6
Documentation website or wiki	28
Stack Exchange / Stack Overflow tag	5
ROS Answers integration	4

GitHub Issues is the universal support channel, used by all 28 packages. Dedicated documentation websites are also universal. Community-oriented channels (forums, Discord servers, mailing lists) show more variation: 9 packages maintain official forums, 8 maintain mailing lists, and 6 maintain Discord servers. This variation partly reflects project age and governance model—newer packages and those stewarded by companies or large labs tend to favor Discord and GitHub Discussions, while older academic packages more commonly use mailing lists.

The overall surface usability impression scores range from 5 to 10, with a mean of 7.3. Gazebo (10) and Drake (9), MoveIt! (9) score highest; they have comprehensive documentation suites and multiple support channels. Newton Dynamics (5) and GraspIt! (5) score lowest.

5.7 Maintainability

Maintainability assesses the effort required to modify a software system or component. Table 5.5 summarizes key maintainability indicators.

Table 5.5: Maintainability indicators across 28 packages.

Indicator	**Count**	**%**	**Notes**
Version number documented	28	100%	—
Code review / contribution guide	26	93%	—
Non-code artifacts present	28	100%	All have config, docs, etc.
Issue tracking via git/GitHub	28	100%	—
Version control via git/GitHub	28	100%	—

All 28 packages use git for version control and host their repositories on GitHub (with the exception of ODE, which uses BitBucket alongside a GitHub mirror). All 28 packages use GitHub Issues or the equivalent for issue tracking. This uniformity is partly a selection effect—the Domain X methodology expresses a preference for GitHub-style repositories because they facilitate consistent measurement—but it also reflects the near-universal adoption of git and GitHub in the contemporary robotics software community.

The percentage of identified issues that are closed ranges from 34.6% (Flightmare) to 100% (DART, Newton Dynamics), with a mean of 79.8% and a median of 82.3%. A high closure rate can indicate active issue management, but it can also reflect a low rate of new issue reporting or a practice of closing stale issues aggressively. These percentages should be interpreted with caution.

The percentage of code that consists of comments ranges from 0.0% (ROS 2, which is measured as zero code lines due to its multi-repository structure where the core communication libraries are compiled from source) to 27.98% (OROCOS), with a mean of approximately 14.2% excluding the ROS 2 outlier. OROCOS (27.98%), OMPL (27.42%), OpenRTM-aist (26.34%), MoveIt! (26.12%), and Simbody (25.27%) lead in comment density. Webots (1.93%), GraspIt! (2.05%), and MORSE (2.67%) have the lowest comment percentages. The Domain X methodology treats 10% as a threshold for the maintainability impression calculator.

The overall maintainability impression scores range from 4 to 10, with a mean of 7.4. Gazebo (10), ROS 2 (9), OMPL (9), Pinocchio (9), and MoveIt! (9) score highest. GraspIt! (4) and Flightmare (4) score lowest, weighed down by their dead or very low activity status and limited contribution infrastructure.

5.8 Reusability

Reusability assesses the extent to which software components can be used in problem solutions other than the one for which they were originally developed. The Domain X template measures reusability through two indicators: the number of code files (as a rough proxy for component granularity) and the presence of API documentation.

The number of code files varies widely across the study set, from 221 (OROCOS) to 7,823 (Newton Dynamics), with a median of approximately 1,500. All 28 packages provide API documentation, typically generated through Doxygen for C++ codebases or Sphinx for Python-heavy projects.

The overall reusability impression scores range from 5 to 10, with a mean of 7.5. MuJoCo (9), OMPL (9), Pinocchio (9), and MoveIt! (9) score highest. These

packages are designed as libraries for integration into larger systems. MORSE (5) and GraspIt! (5) score lowest.

5.9 Surface Understandability

Surface understandability is assessed through a review of 10 randomly selected source files per package, examining code formatting, identifier quality, comment practices, and modularization. This is a shallow check, not a comprehensive code review.

Table 5.6 summarizes the surface understandability indicators.

Table 5.6: Surface understandability indicators across 28 packages.

Indicator	**Packages**	**%**
Consistent indentation and formatting style	28	100%
Explicit identification of a coding standard	10	36%
Consistent, distinctive, and meaningful identifiers	28	100%
Hard-coded constants (other than 0 and 1) present	28	100%
Comments are clear and describe what is being done	28	100%
Name/URL of algorithms used is mentioned	27	96%
Code is modularized	28	100%

All 28 packages exhibit consistent indentation and formatting, meaningful identifiers, clear comments, and modularized code organization. These are baseline indicators of professional software development practice, and their universal presence is a positive signal for the domain.

Only 10 of 28 packages (36%) explicitly identify a coding standard. This is consistent with findings in other domains: the MI study **?** found that coding standards were rarely documented. Packages that do identify a coding standard include Gazebo, ROS 2, MuJoCo, Drake, DART, OMPL, Pinocchio, and a few others. These tend to be projects with formal governance structures and explicit contributor guidelines.

The overall surface understandability impression scores range from 5 to 10, with a mean of 7.4. Gazebo (10), OMPL (9), Drake (9), and MoveIt! (9) score highest. GraspIt! (5) scores lowest.

5.10 Visibility and Transparency

Visibility and transparency assess the extent to which a project's development process and status are visible to external observers. Table 5.7 summarizes the key indicators.

Table 5.7: Visibility and transparency indicators.

Indicator	**Packages**	**%**
Development process defined (CI, contributing guide, etc.)	26	93%
Documents recording development process and status	28	100%
Development environment documented	28	100%
Release notes published	27	96%

Twenty-six of 28 packages have a defined development process, typically manifested through CI/CD workflows, contributing guidelines, pull request templates, and issue templates. All 28 packages use GitHub Issues or similar tools to record

development status. All 28 document their development environment (build dependencies, toolchain requirements, etc.). Twenty-seven of 28 publish release notes.

The two packages without a clearly defined development process are CoppeliaSim (whose mixed licensing model means parts of the development process are not publicly visible) and ODE (whose low activity level means formal process documentation is minimal).

The overall visibility and transparency impression scores range from 5 to 10, with a mean of 7.5. Gazebo (10), Drake (9), ROS 2 (9), OMPL (9), and MoveIt! (9) score highest. These packages publish thorough documentation of their development workflows. MORSE (5), Flightmare (5), Newton Dynamics (5), and GraspIt! (5) score lowest, consistent with their dead or low-activity status.

5.11 Repository Metrics

Table 5.8 presents key repository metrics for all 28 packages, including GitHub popularity indicators and codebase size measures. All data is from May 2026.

Table 5.8: Repository metrics for the 28 selected packages (May 2026).

Software	**Stars**	**Forks**	**Commits**	**Code Lines**	**% Comments**	**% Issues Closed**
AirSim	18,159	4,889	3,562	134,331	10.3%	80.0%
Bullet/PyBullet	14,465	3,074	9,405	1,528,236	2.8%	87.3%
CARLA	13,941	4,559	6,713	138,801	9.4%	82.0%
ROS 2	5,475	903	309	—	—	86.2%
MuJoCo	13,440	1,501	4,676	349,490	6.7%	91.7%
Drake	4,025	1,366	34,786	705,686	20.5%	90.0%
Webots	4,345	2,025	15,746	581,442	1.9%	87.9%
Gazebo	1,337	411	7,500	193,808	11.9%	56.7%
PhysX (OSS)	4,532	614	4,662	1,030,524	17.3%	67.9%
Pinocchio	3,354	537	13,081	185,472	11.9%	92.3%
Habitat	3,662	530	1,664	143,150	7.7%	75.7%
Proj. Chrono	2,836	595	19,510	657,303	1.1%	96.8%
Simbody	2,521	492	5,041	259,447	25.3%	59.5%
MRPT	2,133	658	10,231	504,916	9.4%	95.4%
OMPL	2,047	690	5,341	105,532	27.4%	93.2%
MoveIt!	1,803	744	9,462	159,789	26.1%	80.5%
Flightmare	1,353	400	139	14,543	20.1%	34.6%
SOFA	1,194	352	27,404	349,961	4.5%	64.1%
DART	1,081	300	6,727	339,389	9.2%	100.0%
CoppeliaSim	146	46	1,206	292,460	5.1%	95.5%
OROCOS	879	443	1,229	18,063	28.0%	78.0%
OpenRAVE	801	355	10,375	623,306	19.2%	42.9%
YARP	590	214	19,020	843,982	15.1%	82.3%
ODEs	—	—	2,119	147,055	18.8%	—
GraspIt!	213	86	736	110,614	2.1%	56.6%
OpenRTM-aist	24	14	4,360	86,965	26.3%	81.4%

Notes: ROS 2 code line and comment counts are not directly comparable because the ROS 2 organization spans hundreds of repositories. ODE is hosted on BitBucket and does not report GitHub-style star and fork counts. The "code lines" column reports SCC-measured code lines in the primary measured repository.

GitHub stars—a rough proxy for community attention—span four orders of magnitude, from 24 (OpenRTM-aist, Newton Dynamics) to 18,159 (AirSim). The median is approximately 2,300. The correlation between stars and measurement scores is not straightforward. AirSim leads in stars by a wide margin but does not appear among the top-scoring packages in most quality categories. Conversely, Gazebo scores highly across most quality categories but has relatively modest star counts (1,337), likely because its community engagement predates the current GitHub repository organization and because its user base interacts with it primarily through ROS rather than directly through its GitHub page.

The number of commits ranges from 139 (Flightmare, which is now inactive) to 34,786 (Drake). Commit activity in the last 12 months shows a similar range: Drake leads with sustained high monthly activity, while several dead packages (MORSE, GraspIt!, Flightmare) show zero or near-zero recent commits.

The percentage of code that is comments shows substantial variation, from 1.1% (Project Chrono) to 28.0% (OROCOS). The Domain X maintainability calculator uses 10% as a threshold: 20 of 28 packages (71%) exceed this threshold. The packages below 10% include some of the largest codebases in the study (Bullet/PyBullet at 2.8%, SOFA at 4.5%, Project Chrono at 1.1%), suggesting that codebase size alone does not determine comment density.

5.12 Overall Observations

Table 5.9 summarizes the overall impression scores across all nine quality categories for each package, along with a simple unweighted mean. The mean is presented as a summary statistic and is not a substitute for the AHP-based ranking, which accounts for differential importance weights across qualities.

Table 5.9: Overall impression scores by quality category (1–10 scale). I = Installability, C = Correctness, SR = Surface Reliability, SRo = Surface Robustness, SU = Surface Usability, M = Maintainability, Re = Reusability, SUnd = Surface Understandability, VT = Visibility/Transparency.

Software	**I**	**C**	**SR**	**SRo**	**SU**	**M**	**Re**	**SUnd**	**VT**
AirSim	8	8	8	8	8	8	8	7	7
Bullet/PyBullet	9	7	8	7	7	7	8	7	7
CARLA	7	8	8	7	8	8	8	7	8
CoppeliaSim	8	8	8	7	7	7	8	7	7
DART	8	8	8	7	7	8	8	7	7
Drake	7	10	10	9	9	9	8	9	9
Flightmare	7	6	7	8	6	4	6	6	5
Gazebo	10	10	10	10	10	10	10	10	10
GraspIt!	5	5	6	5	5	4	5	5	5
Habitat	8	8	8	8	8	8	8	8	8
MORSE	9	8	8	8	8	8	5	6	5
MoveIt!	7	9	9	8	9	9	9	9	9
MRPT	8	8	7	8	7	8	8	8	8
MuJoCo	7	7	8	8	8	8	9	8	8
Newton Dyn.	6	5	6	6	5	5	6	6	5
ODE	4	6	7	7	6	6	7	7	7
OMPL	9	9	9	8	8	9	9	9	9
OpenRAVE	8	8	8	7	8	8	7	8	7
OpenRTM-aist	7	8	7	7	6	7	6	7	7
OROCOS	7	8	7	6	6	6	7	6	6
PhysX (OSS)	4	6	6	6	7	8	8	8	8
Pinocchio	10	9	9	8	8	9	9	8	8
Proj. Chrono	7	8	8	7	7	8	7	7	8
ROS 2	8	9	9	8	9	9	9	8	9

Several patterns emerge from the measurement results:

High performers across multiple qualities. Gazebo scores 10 in all nine categories, making it the most consistently high-rated package in the study. Drake and MoveIt! score 9 or 10 across most categories. ROS 2, OMPL, and Pinocchio also show strong scores across the board. These packages share several characteristics: they are stewarded by established organizations or research groups, have active contributor communities, maintain comprehensive documentation, use automated testing and CI/CD, and publish clear contribution guidelines and release notes.

Dead or inactive packages score lower across the board. GraspIt! (last commit July 2020), Flightmare (last commit May 2023), and MORSE (last commit March 2022) appear in the bottom tier for most quality categories. This is expected: packages that are no longer maintained tend to accumulate outdated documentation, broken installation procedures, and unresolved issues. Newton Dynamics, while technically alive (last commit May 2026), scores in the bottom tier, reflecting its status as a primarily individual-maintained project with limited documentation infrastructure.

Installability is generally strong. All 28 packages provide installation instructions and automation, and only 2 broke during installation. This compares favorably with prior State-of-the-Practice studies. The widespread adoption of package managers (APT, pip, Conda), containerization (Docker), and standardized build systems (CMake) in the robotics community likely contributes to this result.

Documentation is universally present but varies in depth. All packages provide tutorials, user manuals, and API documentation. However, only one package documents expected user characteristics, and only 10 explicitly identify a coding standard. These gaps are consistent with findings in medical imaging and LBM

software **??** and reflect broader patterns in research software development.

Repository activity is concentrated in a subset of packages. A small number of packages (Drake, SOFA, Project Chrono, YARP, Pinocchio) account for a disproportionate share of total commits. This reflects sustained multi-year development by well-resourced teams. At the other extreme, several packages show minimal activity in the last 12 months.

The relationship between community popularity and measurement scores is not linear. AirSim leads in GitHub stars by a wide margin (18,159) but scores in the middle tier (7–8) across most quality categories. This is consistent with the finding in the MI study **?** that GitHub popularity and measurement-based rankings do not always align. Popularity metrics are cumulative and backward-looking, while measurement scores reflect current documentation and process quality.

These observations are descriptive and are based on the specific measurement procedure, time frame, and interpretation rules described in Chapter 3. They are not claims that any single package is universally superior. Chapter 7 interprets these results in the context of the research questions and discusses threats to validity.

Chapter 6

Developer Interviews

Developer interviews are part of the Domain X State-of-the-Practice methodology. They provide qualitative evidence that is difficult to obtain from repositories alone, such as developer pain points, project organization, documentation practices, and the reasons behind particular quality-related decisions. In this thesis, the interview chapter is kept separate from the measurement results because the two sources of evidence answer related but different questions.

At the current drafting stage, no interview findings are reported. This chapter records the planned role of the interview component and the structure that should be used if approved interview data becomes available.

6.1 Purpose of the Interviews

Repository and documentation measurements can show visible project artifacts, but they cannot always explain why a team made particular development choices. Interviews can help interpret gaps in public evidence. For example, a project may use

testing, continuous integration, documentation practices, or informal review processes that are not clearly visible from the repository. Interviews can also identify obstacles that developers face when maintaining research software.

The interview component is expected to support the research questions about developer pain points, software quality practices, documentation, maintainability, understandability, usability, and reproducibility. It should complement the measurement template rather than replace it.

6.2 Recruitment and Ethics

The Domain X methodology recommends contacting developers from the selected software packages and using the same recruitment approach across the study set to reduce selection bias. In this project, interview recruitment and reporting must follow the applicable McMaster ethics approval and consent requirements.

No participant responses, identifying details, or interview findings should be included unless the relevant ethics materials, consent records, transcripts, notes, or approved summaries are available.

6.3 Interview Protocol

The Domain X interview guide contains 20 semi-structured questions. These questions cover the interviewee's role, the development team, project users, project organization, development practices, documentation, current and past difficulties, and actions taken to address software qualities such as maintainability, understandability, usability, and reproducibility. The interviews are semi-structured so that follow-up

questions can clarify answers or explore topics raised by participants.

For the robotics software domain, the interview protocol should also preserve the distinction between simulators, planning libraries, middleware, physics engines, and related tools. A question about usability or documentation may have different implications for a full simulator than for a lower-level library.

6.4 Analysis Plan

Interview responses should be analyzed as qualitative data. The analysis should identify recurring pain points, reported development practices, and strategies used by developers to address software qualities. The coding or grouping process should be documented so that the interpretation is traceable.

Interview evidence should be connected back to the repository and measurement-template evidence where possible. If interviews reveal practices that were not visible in public artifacts, the thesis should distinguish between publicly observed evidence and participant-reported evidence.

6.5 Interview Findings

This section is reserved for approved interview findings. Possible subsections include developer background, project organization, documentation practices, quality-related practices, pain points, and developer recommendations.

Chapter 7

Discussion

7.1 Answers to Research Questions

7.2 Interpretation of the Measurement Results

7.3 Implications for Robotics Software Users

7.4 Implications for Software Maintainers

7.5 Threats to Validity

7.5.1 Candidate Discovery

7.5.2 Measurement Consistency

7.5.3 Snapshot-Based Repository Data

7.5.4 Use of LLM-Assisted Candidate Sources

Chapter 8

Recommendations

8.1 Recommendations for Robotics Software Users

8.2 Recommendations for Software Maintainers

8.3 Recommendations for Future State-of-the-Practice Studies

Chapter 9

Conclusions

9.1 Summary

9.2 Key Findings

9.3 Limitations

9.4 Future Work

Appendix A

Full Candidate Software List

Appendix B

Mention Count Source List

Appendix C

Measurement Template

Appendix D

Excluded Software Packages

Appendix E

Interview Materials